Concept Drift Detection on NYC Taxi Demand Streams

Technical report, Big Data mini project, Option A. Team: Rahul Saran Sharma, Rahul Prasanna, Raghav U.

1. Problem and approach

A ride-demand model trained on one period degrades when city travel patterns change. We forecast hourly yellow-taxi pickups for each of the 263 NYC taxi zones one hour ahead, stream 2019-2021 TLC trip data through the model, and ask four questions: (1) which streaming drift detectors find the change, how fast and with how many false alarms; (2) whether detector-triggered retraining beats never retraining or retraining on a schedule; (3) whether drift hurts some boroughs more than others; and (4) whether a city-wide detector is enough to protect every borough.

The model is a single LightGBM regressor over all zones with lag features (1, 2, 24 and 168 hours), 24 h and 168 h rolling means, hour of day, day of week, a holiday flag and the zone id. It is trained on January-June 2019; July-September 2019 is used for early stopping and to calibrate the detectors; October 2019 onward is the stream under test. On October-December 2019 it reaches a city-wide WAPE of **13.1%** against **22.0%** for a seasonal-naive baseline (same hour last week).

2. Pipeline

A replay producer reads the monthly Parquet files and publishes trips to the Kafka topic *trips* in event-time order (3600x real time). Spark Structured Streaming groups them into 1-hour tumbling windows per pickup zone with a 2-hour watermark, writes the counts to the *zone_demand* topic and to partitioned Parquet. The monitor service consumes zone-hour counts, predicts, scores the error, and feeds two signals to the detectors: the hourly city (or borough) WAPE divided by its July-September value for the same hour of week, and total demand divided by its calibrated hour-of-week value. Normalising by hour of week removes the daily and weekly cycle so the detectors see a roughly stationary signal when nothing has changed. A drift event goes to the retraining controller and to an append-only audit log; promoted models are versioned in MLflow.

In the live run we replayed March 2020 end to end through Kafka and Spark on a laptop: 3,245,202 trips at a median of 1,819 rows per second, 288 stream hours, 7 alarms in total (city and borough). The city PSI warned on 2020-03-08 07:00:00; city ADWIN fired on 2020-03-12 23:00:00, which the combined policy accepted as confirmed drift and retrained. The challenger scored 20.6% WAPE on the 48-hour holdout against 17.9% for the champion, so the gate kept the champion (retrain took 299 s).

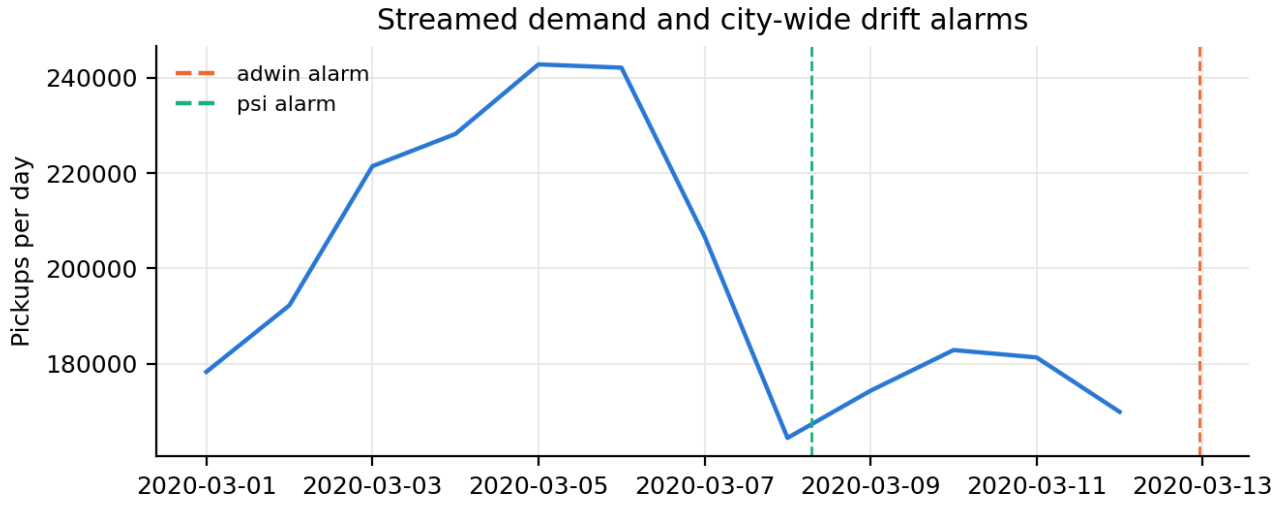


Figure 1. Live run, March 2020: city error signal, demand and detector alarms as produced by the Kafka - Spark - monitor pipeline.

3. Detectors and benchmark

All detectors share one interface (*update(x) -> drift?*) and are implemented from scratch: **ADWIN** (adaptive window, cuts when two sub-windows differ by more than a Hoeffding bound with confidence delta), **PSI** (Population Stability Index of the last 168 hours against a 10-bin reference histogram, evaluated every 6 hours, warning at 0.4x the drift threshold), **Page-Hinkley** (cumulative deviation from the running mean) and **KSWIN** (Kolmogorov-Smirnov test between a recent and a reference window). ADWIN, Page-Hinkley and KSWIN watch the model error; PSI and a second ADWIN watch demand.

Real data has no labelled change points, so detection delay is measured on a synthetic benchmark: we inject drift with a known start into October-December 2019 (abrupt -50%, abrupt +50%, a 2-week gradual ramp to -50%, and a 3-hour shift of the daily profile), into the whole city or one borough, with 8 random change points each. Delay is hours from the true change to the first alarm (capped at 336 h); false alarms are alarms before the change and on a no-drift stream, per 1000 stream hours.

Detector	Signal	Detection rate	Median delay (h)	False alarms / 1000 h
page_hinkley_t5	error	100%	2.0	11.21
page_hinkley_t10	error	100%	7.0	5.26
page_hinkley_t20	error	100%	17.0	2.26
adwin_d0.002	error	100%	26.5	4.42
adwin_d0.01	error	100%	24.5	4.81
adwin_d0.05	error	100%	23.5	6.85
adwin_demand_d0.002	demand	100%	38.5	3.62
kswin_a0.0005	error	100%	28.5	10.70
kswin_a0.005	error	100%	37.0	12.40
psi_t0.1	demand	3%	171.0	0.45
psi_t0.25	demand	6%	182.0	0.45
psi_t0.5	demand	12%	194.0	0.45

Table 1. Detector benchmark, all scenarios and targets pooled.

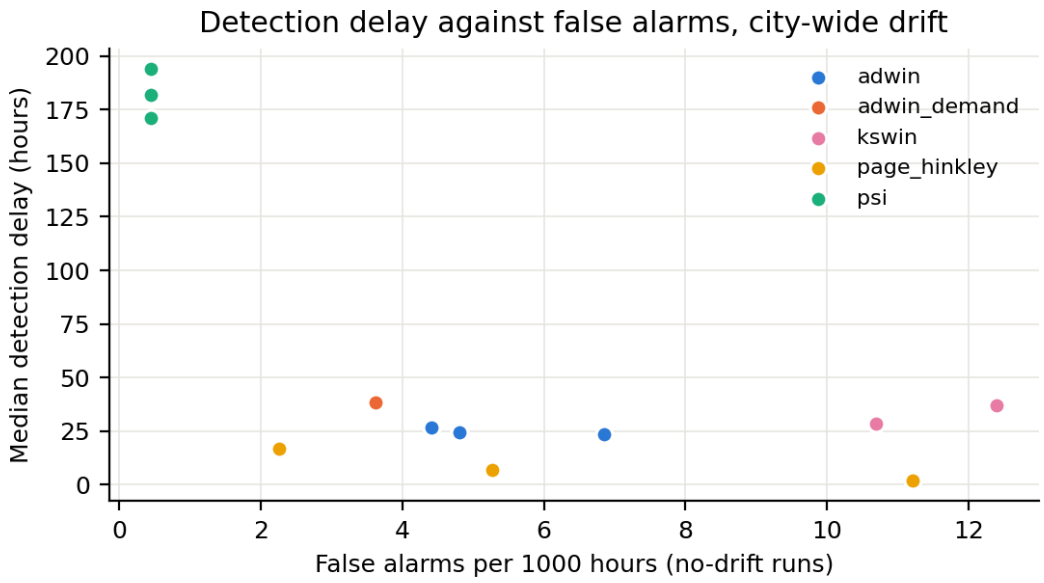


Figure 2. Speed against false alarms for every detector setting.

Findings. Page-Hinkley is the fastest (median 2 h at threshold 5) but pays for it with 11.2 false alarms per 1000 h; at threshold 20 it takes 17 h with 2.3. ADWIN (delta 0.002) finds every injected change in a median 26.5 h with 4.4 false alarms per 1000 h and has no threshold to hand-tune. KSWIN is no faster than ADWIN and raises 2-3 times as many false alarms. PSI hardly ever false-alarms, but with a 168-hour window it only reached the threshold on the gradual scenario (29% of runs, ~180 h late): a 50% change in total demand moves the binned distribution of the normalised demand ratio less than the thresholds tuned on real data allow. PSI is therefore useful as a slow confirmation signal, not as a primary detector.

Real drift (COVID-19). On the real stream, ADWIN first fired after 1 March on 2020-03-12 23:00:00, 49 hours before the 15 March demand collapse usually quoted, with 12 alarms in the five calm months before it; Page-Hinkley (t=20) fired on 2020-03-12 21:00:00 after 7 earlier alarms. KSWIN and Page-Hinkley at low thresholds alarmed throughout early March and February, so their 'early' alarms are not informative. City PSI did not fire in the March window at all in the offline run.

4. Retraining policy

Every policy is replayed over October 2019 - December 2021 with the same controller. On a trigger it trains a challenger on the last 6 weeks, compares it with the champion on the 48 hours just before the trigger (which the challenger does not train on), and promotes it only if its WAPE is lower. A 72-hour cooldown stops alarm storms from causing back-to-back retrains. Policies: *never*; *periodic* (first of every month); *adwin* (city ADWIN on error); *psi*; *combined* (ADWIN confirmed by a PSI warning within 7 days); *adwin_any_borough* (city ADWIN or any borough's ADWIN); and *oracle*, which retrains on six known real change dates and serves as a reference.

Policy	WAPE before	WAPE crash	WAPE recovery	Retrains	Promoted	Train time (s)
never	13.15	28.87	18.35	0	0	0.0
periodic	13.83	36.32	19.04	27	22	61.3
adwin	13.15	30.42	19.22	34	17	75.0
psi	13.15	28.87	18.35	1	0	2.2
combined	13.15	30.42	19.22	34	17	75.5
adwin_any_borough	13.15	28.56	18.87	58	31	137.9
oracle	13.15	29.89	22.92	6	6	11.9

Table 2. City-wide WAPE (%) by period and cost of each policy. Before = Oct 2019-Feb 2020, crash = Mar-Jun 2020, recovery = Jul 2020-Dec 2021.

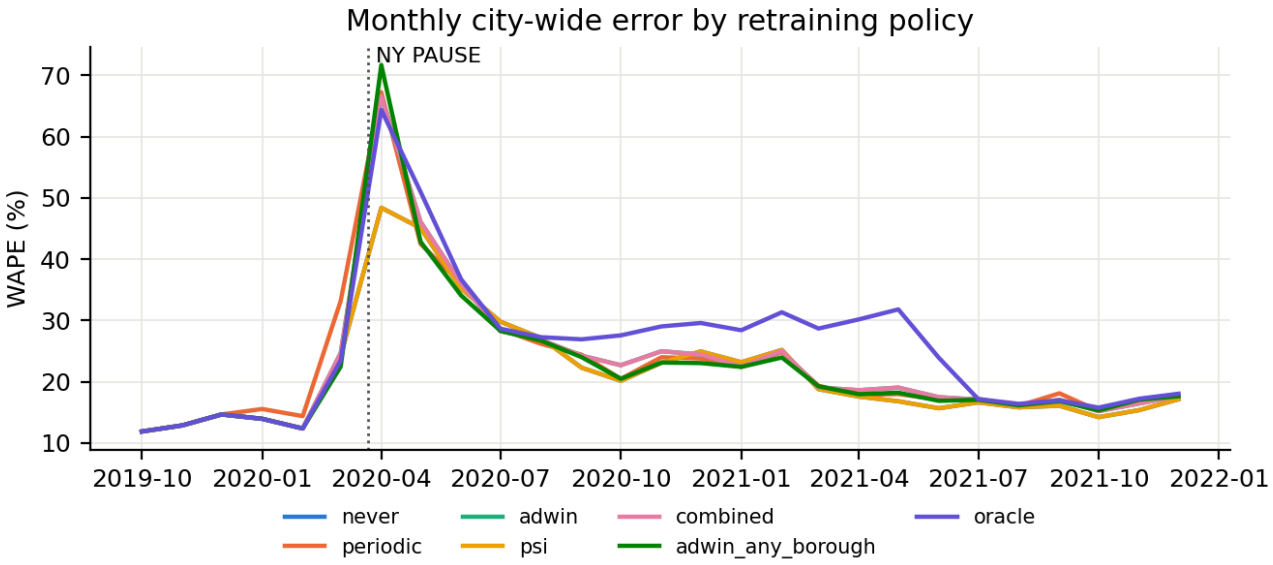


Figure 3. Daily city WAPE for each retraining policy across 2019-2021.

Findings. The best crash-period error comes from *adwin_any_borough* (28.56% vs 28.87% for never retraining), and periodic retraining is clearly worst (36.32%) because it retrains on calendar dates that straddle the collapse. The honest result, however, is that the gains are small and *never* retraining is best during recovery (18.35%). The model already adapts through its 1- and 2-hour lag features, and a 6-week window that contains the abnormal spring of 2020 teaches the model patterns that no longer hold later. The holdout gate is what keeps detector-triggered retraining safe: only 31 of 58 challengers were promoted, and in the live run the gate rejected a challenger that was worse than the champion. Even the oracle does not beat never retraining, which says the bottleneck is the retraining recipe (window length, features), not the trigger.

Recommended policy. Trigger on city ADWIN on the error signal (delta 0.002) *or* any borough's ADWIN; require the challenger to beat the champion on a 48-hour holdout; 72-hour cooldown; 6-week training window; log every decision. Two changes we would make next, based on Section 5: gate on per-borough error as well as city error,

and use a longer or weighted training window after an abrupt shift so the model is not retrained only on the shock.

5. Does drift hurt some boroughs more?

Borough	WAPE before	WAPE crash	Crash / before	Crash WAPE, never	Crash WAPE, adwin_any_borough
Manhattan	11.8	24.6	2.09x	24.6	22.6
Queens	21.8	52.5	2.41x	52.5	59.9
Brooklyn	56.8	100.5	1.77x	100.5	124.0
Bronx	114.3	131.5	1.15x	131.5	169.4
Staten Island	211.5	215.7	1.02x	215.7	1154.8

Table 3. Borough WAPE (%) for the never-retrained model, and crash-period WAPE under the best policy.

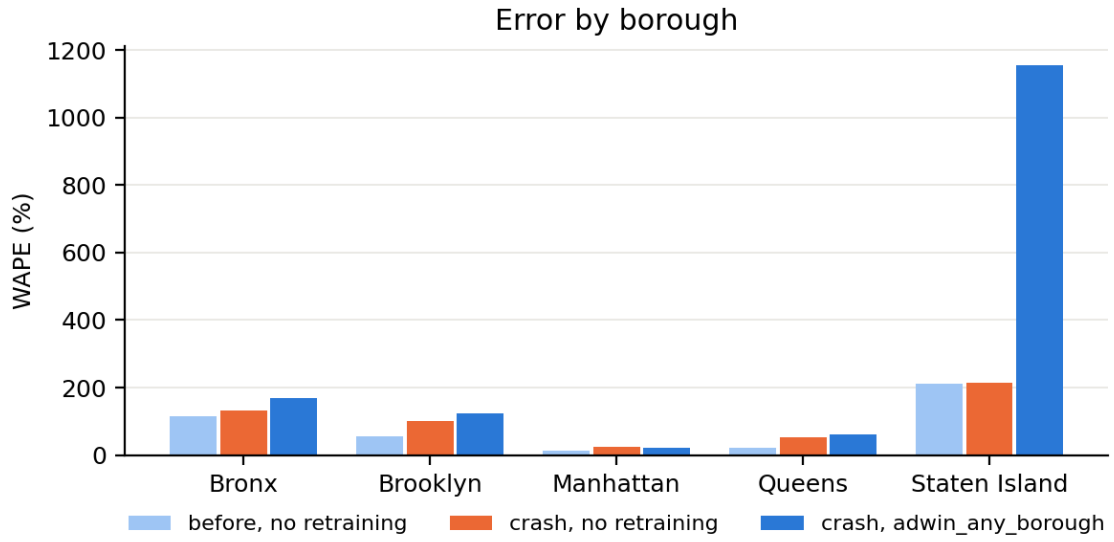


Figure 4. Borough error before, during and after the COVID crash.

Yes, unevenly. Relative to its own baseline, Queens was hit hardest (error x2.41), driven by the collapse of airport trips at JFK and LaGuardia, then Manhattan (x2.09) and Brooklyn (x1.77). The Bronx and Staten Island barely moved because their yellow-taxi demand is so small that the model was already poor there (WAPE above 100%, often predicting a fraction of a trip per zone-hour). Retraining makes the gap worse: under every policy that promoted a new model, Staten Island's crash-period WAPE jumps from 216% to over 1155%, because the city-level gate accepts a model that is better for Manhattan, which dominates the total, and worse for tiny boroughs. This is why the recommended gate checks each borough.

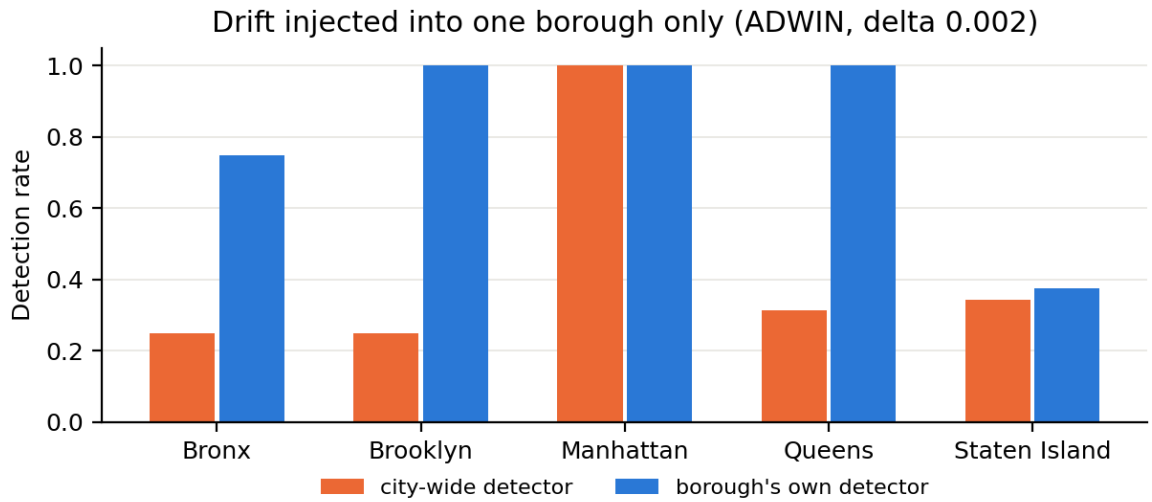


Figure 5. Share of borough-only drift caught by the borough's own detector against the city-wide detector.

A city-wide detector does not protect every borough. When drift is injected into one borough only, city ADWIN catches it in Manhattan every time but in Queens only 31% of runs and in the Bronx 25%, while each borough's own ADWIN catches 100% and 75%. City PSI caught none of the outer-borough changes (Brooklyn: 0% city vs 50% own). Per-borough detectors are cheap and are part of the recommended policy.

6. Limitations

Delay is measured on synthetic drift because real changes have no exact start; COVID is used only as a qualitative check. WAPE in the Bronx and Staten Island is dominated by near-zero counts and should be read as 'poor' rather than as a precise number. Yellow taxis are a shrinking share of outer-borough trips, so borough results describe yellow-taxi demand, not mobility. The Docker Compose stack is written but was not run; the tested setup is native Linux/WSL with a single Kafka broker and Spark in local mode, so we make no claims about cluster scaling. The live run covers one month of replay.

7. Reproducing

scripts/pipeline.sh setup | *data* | *up* | *demo* builds the environment, downloads the data, trains the model and runs the live demo; *experiments/run_detectors.py* and *experiments/run_retraining.py* regenerate every table here into *results/*; *notebooks/results.ipynb* rebuilds the figures and *scripts/build_report.py* rebuilds this report from *results/summary.json*.